

Digital Electronics

Part 1 - Binary Representation of

Information

Stéphane Le Goff

2026

1. Binary Representation of Information

The decimal system is the most familiar number system to us because we use it every day. It is composed of ten symbols, which are 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. Using these symbols as digits of a number, we can express any quantity. The decimal system is also called the base-10 system because it has ten digits.

Example: $(1672.63)_{10} = 1 \times 10^3 + 6 \times 10^2 + 7 \times 10^1 + 2 \times 10^0 + 6 \times 10^{-1} + 3 \times 10^{-2}$

10^3	10^2	10^1	10^0		10^{-1}	10^{-2}
1	6	7	2	.	6	3
MSD				DP		LSD

MSD: most significant digit, DP: decimal point, LSD: least significant digit

Digital systems use the binary system in which there are only two symbols or digit values, defined by convention to be 0 and 1. This base-2 system can be used to represent any quantity, exactly like the base-10 system.

Example: $(1011.01)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} = 8 + 2 + 1 + 0.25 = (11.25)_{10}$.

2^3	2^2	2^1	2^0		2^{-1}	2^{-2}
8	4	2	1		0.5	0.25
1	0	1	1	.	0	1
MSB				BP		LSB

MSB: most significant bit, BP: binary point, LSB: least significant bit

In more general and formal mathematical terms, we can state that any integer decimal number A can be represented in base 2 by a vector (or sequence), of n binary digits (bits) $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ such that

$$A = a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0 = \sum_{i=0}^{n-1} a_i 2^i.$$

The same expression can be adapted to the definition of any fractional decimal number

$A = (a_{n-1} \dots a_0.a_{-1} \dots a_{-m})_2$ such that

$$A = a_{n-1}2^{n-1} + \dots + a_02^0 + a_{-1}2^{-1} + \dots + a_{-m}2^{-m} = \sum_{i=-m}^{n-1} a_i 2^i.$$

This type of binary representation of a decimal number is known as the *natural binary representation*. Using such binary representation, it is possible to process any set of quantities exactly as we do with the decimal system.

We have shown how to represent a decimal number in its binary equivalent. This method is undoubtedly the most logical one and this is why it is referred to as the *natural binary representation*. However, it suffers from some inherent limitations such as the inability to consider negative numbers, as will be discussed at length later in this document.

Using the binary representation of numbers, we can implement all arithmetic operations in binary: addition, subtraction, multiplication, and division. We are now going to show several examples to illustrate this point.

• Arithmetic operations on unsigned binary numbers

Unsigned binary numbers are binary numbers that can only be positive, i.e. greater than or equal to zero. They are most often chosen to be integers by digital system designers.

With unsigned numbers, we employ the natural binary representation. In fact, there exist other ways to convert a decimal number into a group of n bits. The natural binary representation is almost always used to encode positive (unsigned) numbers. However, other representations, such as the two's complement representation, must be used when encoding signed numbers that are defined as numbers that can be either positive or negative. We shall come back to this important point later.

For example, we have shown in the table below the 16 integer values that can be generated by using four bits. For each binary number, we have also shown its equivalent in decimal (base 10) and in hexadecimal (base 16).

4-bit unsigned number	Decimal number	Hexadecimal number
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	10	A
1011	11	B
1100	12	C
1101	13	D
1110	14	E
1111	15	F

Note that we have also mentioned in this table the existence of the hexadecimal system, which is a very convenient type of representation often encountered in digital electronics. It is convenient since it does offer the possibility to represent any 4-bit number using a single symbol ranging from 0 to F.

With n bits, we can generate 2^n different numbers that, if we only consider integers, range from 0 to 2^n-1 in decimal.

For example, with $n = 8$, we can generate all integer values ranging from $(0)_{10}$ (i.e., 00000000 in binary or 00 in hexadecimal) to $(255)_{10}$ (i.e., 11111111 in binary or FF in hexadecimal).

We have indicated, in the table below, the number of different values as well as the associated integer range that can be generated for several values of n .

One can see that it does not take a large number of bits to generate quite a lot of possibilities. This is the magic of the exponential growth.

Number of bits	Number of different values that can be generated	Range of integer numbers
1	2	0 – 1
2	4	0 – 3
3	8	0 – 7
4	16	0 – 15
5	32	0 – 31
6	64	0 – 63
7	128	0 – 127
8	256	0 – 255
9	512	0 – 511
10	1,024	0 – 1,023
11	2,048	0 – 2,047
12	4,096	0 – 4,095
13	8,192	0 – 8,191
14	16,384	0 – 16,383
15	32,768	0 – 32,767
16	65,536	0 – 65,535

Modern microprocessors typically manipulate binary words that are 64-bit long. Those words can thus take any integer value in the range from 0 to $2^{64}-1 \sim 1.84467441 \times 10^{19}$. That undoubtedly leaves us with an astronomical number of possibilities.

Let us now turn our attention to the addition of two unsigned binary numbers A and B.

Addition of unsigned numbers

Consider the example below in which we add two 5-bit numbers: $A = (10110)_2$ (22 in decimal, 16 in hexadecimal) and $B = (10111)_2$ (23 in decimal, 17 in hexadecimal).

Carry	1	0	1	1	0	0
A		1	0	1	1	0
B		1	0	1	1	1
S = A + B	1	0	1	1	0	1

We can see that $S = A + B = (101101)_2$. In decimal, we would obtain $S = (22)_{10} + (23)_{10} = (45)_{10}$. We obtain the same result in both cases since 101101 in binary is equivalent to 45 in decimal, i.e. $(101101)_2 = (45)_{10}$.

Note that, when adding two n -bit numbers A and B , we can obtain a result $S = A + B$ that is $(n+1)$ -bit long. The sum S in decimal can indeed range from 0 to $2^n - 1 + 2^n - 1 = 2^{n+1} - 2$, which demonstrates the possible need to use $(n+1)$ bits to represent S in binary. The extra space required to store an additional bit may be an issue if there is only space for storing n bits in our system.

Subtraction of unsigned numbers

Let us now consider the subtraction operation. But, first, we need a quick reminder on how to subtract two n -digit numbers A and B in decimal.

Consider the following subtraction shown below.

A		6	5	8	9	2
B		3	0	1	5	6
Borrow	0	0	0	0	1	0
Difference $D = A - B$		3	5	7	3	6

In this example, the fact that the final borrow equals zero indicates that $A \geq B$, and the result is thus correct.

Now, consider another subtraction:

A		3	0	1	5	6
B		6	5	8	9	2
Borrow	1	1	1	1	0	0
$D = A - B$		6	4	2	6	4

In the example above, the final borrow is equal to one, which indicates that $A < B$, and the result is therefore wrong. This erroneous result simply comes from the fact that the difference $D = A - B$ is negative, something that cannot be supported by an unsigned representation.

There exists a way to obtain the correct difference. We can proceed as follows: determine the ten's complement of D and insert a negative sign in front of this new number.

The ten's complement of an n -digit decimal number X is the decimal number $10^n - X$. For instance, the ten's complement of 64,264 is given by $10^5 - 64,263 = 35,736$.

Note that the ten's complement of any number can also be obtained by computing its nine's complement and adding one to it. Here, the nine's complement of 64,264 is 35,735, to which we add one, thus yielding 35,736.

The correct result of the subtraction is thus -35,736.

A		3	0	1	5	6
B		6	5	8	9	2
Borrow	1	1	1	1	0	0
Incorrect difference		6	4	2	6	4
Nine's complement		3	5	7	3	5
Nine's complement + 1		3	5	7	3	6
Correct $D = A - B$	-	3	5	7	3	6

Once again, note that, although we can find the right answer by finding the ten's complement of the initial erroneous result, doing so might not be very useful in a practical system as the unsigned representation does not allow for the insertion of a minus sign to the result. In practice, we probably have to stick to the original result and simply warn the rest of the circuit that the latter is negative and thus incorrect as such by activating an N flag (N as negative).

Now, to subtract two n-bit binary numbers, we must follow a similar procedure. Consider the first example below where $B = (10011)_2$ is subtracted from $A = (10110)_2$.

A		1	0	1	1	0
B		1	0	0	1	1
Borrow	0	0	0	1	1	0
Difference $D = A - B$		0	0	0	1	1

One can see that the final borrow is equal to zero, thus indicating that the result is positive, and implying that A is greater than or equal to B . So, the result is a positive difference $D = A - B = (00011)_2$. In decimal, we would obtain $D = A - B = (22)_{10} - (19)_{10} = (3)_{10}$. Everything is making perfect sense.

Consider now the subtraction $A - B = (10011)_2 - (10110)_2$ shown below. In decimal, it corresponds to the subtraction $A - B = (19)_{10} - (22)_{10}$. We notice that $A < B$. Therefore, we expect to encounter some issues here.

A		1	0	0	1	1
B		1	0	1	1	0
Borrow	1	1	1	0	0	0
$D = A - B$		1	1	1	0	1

The final borrow equals one. This indicates that $A < B$, and the result is thus incorrect as the natural binary representation of numbers cannot accommodate negative numbers. However, in theory, we could still obtain the correct difference by computing the two's complement of the erroneous result and put a negative sign in front of it.

The two's complement of an n-bit number X is defined as the natural representation of the decimal difference $2^n - X$. Consider, for instance, the binary number $X = (11101)_2$. Its decimal equivalent is $(29)_{10}$, which leads to $2^n - X = 2^5 - X = 32 - 29 = (3)_{10}$. The natural binary representation of $(3)_{10}$ is $(00011)_2$. So, the two's complement of $(11101)_2$ is $(00011)_2$.

The 2's complement of any number can also be obtained, without having to resort to decimal representations, by computing its one's complement, which is very straightforward, and adding one to it. The one's complement of $(11101)_2$ is $(00010)_2$, to which we add $(00001)_2$ to finally obtain $(00011)_2$.

The final and correct result is thus $-(00011)_2$.

A		1	0	0	1	1
B		1	0	1	1	0
Borrow	1	1	1	0	0	0
Incorrect difference		1	1	1	0	1
One's complement		0	0	0	1	0
One's complement + 1		0	0	0	1	1
Correct D = A – B	-	0	0	0	1	1

In decimal, we would obtain $D = (19)_{10} - (22)_{10} = -(3)_{10}$.

But, once again, we must stress that there is no way to represent this correct result in an unsigned representation in which all numbers must, by definition, be positive. So, although we know very well how to obtain the correct result, the fact remains that we cannot represent and process the latter due to the intrinsic limitations of not being able to handle negative numbers.

Performing the subtraction of two unsigned numbers through an addition

It is worth mentioning that one can perform a subtraction by doing an addition instead. It comes from the simple fact that $D = A - B$ can be also written as $D = A + (-B)$. In other words, if we manage to put a negative sign in front of B, then the subtraction operation can be replaced by an addition. The way to invert the number B consists of employing the two's complement representation of B instead of its natural representation.

We have previously seen that the two's complement representation of B is identical to the natural representation of $2^n - B$. Therefore, the binary addition of A and the two's

complement of B yields the natural representation of $A + 2^n - B = 2^n + (A - B)$. The sum is indeed equal to the desired result, which is $A - B$, provided that the term 2^n is discarded.

To illustrate this, let us revisit the two previous examples.

The way to subtract $B = (10011)_2$ from $A = (10110)_2$ by using an addition rather than a subtraction is illustrated in the table below.

B		1	0	0	1	1
2's complement of $B = 2^n - B$		0	1	1	0	1
A		1	0	1	1	0
Carry	1	1	1	0	0	0
$2^n + (-B) + A = 2^n + A - B$	1	0	0	0	1	1
Sum $S = A + (-B) = A - B$	0	0	0	0	1	1

One can see that the final carry in the sum $2^n + A - B$, corresponding to the term 2^n , is equal to one. Discarding the term 2^n simply consists of inverting this final carry. Thus, the result $A - B$ is found to be positive, i.e. correct, since its final carry is zero. So, the result is a positive difference $D = A - B = (00011)_2$. In decimal, we would obtain $D = A - B = (22)_{10} - (19)_{10} = (3)_{10}$.

Consider now the subtraction $A - B = (10011)_2 - (10110)_2$ shown below. We have previously seen that the result should be $(11101)_2$ with a final borrow of one, thus indicating that this result is negative and therefore not consistent with an unsigned number. Do we obtain the same result here? Let us check.

B		1	0	1	1	0
2's complement of $B = 2^n - B$		0	1	0	1	0
A		1	0	0	1	1
Carry	0	0	0	1	0	0
$2^n + A + (-B) = 2^n + A - B$	0	1	1	1	0	1
Sum $S = A + (-B) = A - B$	1	1	1	1	0	1

Here, the final carry, in the sum $2^n + A - B$, corresponding to the term 2^n , is equal to zero. Discarding the term 2^n consists of inverting this final carry. The result $A - B$ is thus found to be negative, i.e. incorrect, because its $(n+1)$ -th bit is equal to one. This is the same result and conclusion as previously.

Multiplication of two unsigned numbers

Let us now turn our attention to the multiplication operation. It is indeed important to show how the multiplication of two n -bit unsigned numbers A and B can be performed. Consider the example below in which we compute the product of $A = (1011)_2$ and $B = (0101)_2$.

A					1	0	1	1
B					0	1	0	1
Row 1					1	0	1	1
Row 2				0	0	0	0	.
Row 3			1	0	1	1	.	.
Row 4		0	0	0	0	.	.	.
Product $P = A \times B$	0	0	1	1	0	1	1	1

One finds that $P = (00110111)_2$. In decimal, we would obtain $P = (11)_{10} \times (5)_{10} = (55)_{10}$. There cannot be any issue here as the multiplication of two positive numbers can only be positive.

For those who do not like to add more than two rows at the same time, it is perfectly acceptable to break the sum operations using intermediate results, as shown below.

A					1	0	1	1
B					0	1	0	1
Row 1					1	0	1	1
Row 2				0	0	0	0	.
Sum Row 1 + Row 2			0	0	1	0	1	1
Row 3			1	0	1	1	.	.
Sum Row 1 + Row 2 + Row 3		0	1	1	0	1	1	1
Row 4		0	0	0	0	.	.	.

Product $P = A \times B$	0	0	1	1	0	1	1	1
--------------------------	---	---	---	---	---	---	---	---

The product of two n -bit numbers can lead to a result that is composed of $2n$ bits. In fact, the product P in decimal can range from 0 to $2^{n-1} \times 2^{n-1} = 2^{2n} - 2^{n+1} + 2$, which demonstrates the possibility of needing $2n$ bits to represent P in binary.

For instance, we have $P = (1111)_2 \times (1111)_2 = (11100001)_2$, as shown below. The extra space required to double the size of the original numbers needs to be taken into consideration in practical systems.

A						1	1	1	1
B						1	1	1	1
Row 1						1	1	1	1
Row 2					1	1	1	1	.
Row 3				1	1	1	1	.	.
Row 4			1	1	1	1	.	.	.
Carry 1		1	1	1	1	1	1	0	0
Carry 2		0	1	1	1	1	0	0	0
Carry 3		0	0	1	1	0	0	0	0
Product $P = A \times B$		1	1	1	0	0	0	0	1

Or, in case you do not like adding more than two rows at the same time:

A						1	1	1	1
B						1	1	1	1
Row 1						1	1	1	1
Row 2					1	1	1	1	.
Row 1 + Row 2				1	0	1	1	0	1
Row 3				1	1	1	1	.	.
Row 1 + Row 2 + Row 3			1	1	0	1	0	0	1
Row 4			1	1	1	1	.	.	.
Product $P = A \times B$		1	1	1	0	0	0	0	1

Division of two unsigned numbers

Finally, let us also study how the division operation is done with two unsigned numbers A and B.

Assume our goal is to divide $A = (1101)_2 = (13)_{10}$ by $B = (0011)_2 = (3)_{10}$. We basically need to do the opposite of the multiplication, i.e. understand that the dividend A is the sum of shifted versions of the divisor B, and find which shifted versions of B are present in A and which versions are not. If a shifted version of B is present, the corresponding position of the division result is a one, else it is a zero.

When dealing with integers, the division of a dividend A by a divisor B leads to both a quotient Q and a remainder R so that we can write $A = Q \times B + R$, as will be made clear by the examples below.

As one can see from the table below, the division process is quite lengthy and more complicated than the other arithmetic operations. Note that the dividend must be extended by three bits equal to zero on the left side. This is needed to give room for the initial subtractions to proceed.

Dividend A			0	0	0	1	1	0	1
Divisor B						0	0	1	1
Subtraction 1			0	0	1	1			
Result of subtraction 1		1	1	1	1	0			
Intermediate quotient						0			
Remainder of A			0	0	0	1	1	0	1
Subtraction 2				0	0	1	1		
Result of subtraction 2			0	0	0	0	0		
Intermediate quotient						0	1		
Remainder of A			0	0	0	0	0	0	1
Subtraction 3					0	0	1	1	
Result of subtraction 3				1	1	1	0	1	
Intermediate quotient						0	1	0	
Remainder of A			0	0	0	0	0	0	1

Subtraction 4						0	0	1	1
Result of subtraction 4					1	1	1	1	0
Final quotient Q						0	1	0	0
Final remainder R						0	0	0	1

The quotient of this division is $Q = (0100)_2 = (4)_{10}$ with a remainder $R = (0001)_2 = (1)_{10}$. We can thus write $A = Q \times B + R = (0100)_2 \times (0011)_2 + (0001)_2 = (4)_{10} \times (3)_{10} + (1)_{10}$.

It is possible to write the result in fractional, rather than integer, form by using, say, two additional bits behind a binary point. To do so, we should continue the division beyond the binary point as shown below.

Note that the bits shown in bold in the table below correspond to the bits that are beyond the binary point.

Remainder of A	0	0	0	1.	0	0
Subtraction 5		0	0	1	1	
Result of subtraction 5	1	1	1	1	1	
Intermediate quotient	0	1	0	0.	0	
Remainder of A	0	0	0	1.	0	0
Subtraction 6			0	0	1	1
Result of subtraction 6		0	0	0	0	1
Final quotient Q	0	1	0	0.	0	1
Final Remainder R	0	0	0	0.	0	1

Finally, the result of the division in fractional form would be $Q = (0100.01)_2 = (4.25)_{10}$. It is not the exact answer as we know from elementary mathematics that the division of $(13)_{10}$ by $(3)_{10}$ yields $(4.33333...)_{10}$. The fact that the final remainder R is not equal to zero clearly indicates that we did not obtain the exact result.

If we needed a more accurate result, we would need to employ more bits beyond the binary point. For instance, by using four bits after the binary point, we would obtain $Q = (0100.0101)_2 = (4.3125)_{10}$. With six bits after the binary point, the result would be $Q = (0100.010101)_2 = (4.328125)_{10}$. Finally, we would obtain $Q = (0100.01010101)_2 = (4.33203125)_{10}$ by using 8 bits beyond the binary point.

Let us consider another example. Our goal is now to divide $A = (11010111)_2 = (215)_{10}$ by $B = (0101)_2 = (5)_{10}$. We must extend the dividend by three MSBs equal to zero to give room for the initial subtractions.

A		0	0	0	1	1	0	1	0	1	1	1
B									0	1	0	1
Subtraction 1		0	1	0	1							
Result of subtraction 1	1	1	1	0	0							
Intermediate quotient					0							
Remainder of A		0	0	0	1	1	0	1	0	1	1	1
Subtraction 2			0	1	0	1						
Result of subtraction 2		1	1	1	1	0						
Intermediate quotient					0	0						
Remainder of A		0	0	0	1	1	0	1	0	1	1	1
Subtraction 3				0	1	0	1					
Result of subtraction 3			0	0	0	0	1					
Intermediate quotient					0	0	1					
Remainder of A		0	0	0	0	0	1	1	0	1	1	1
Subtraction 4					0	1	0	1				
Result of subtraction 4				1	1	1	1	0				
Intermediate quotient					0	0	1	0				
Remainder of A		0	0	0	0	0	1	1	0	1	1	1
Subtraction 5						0	1	0	1			
Result of subtraction 5					0	0	0	0	1			
Intermediate quotient					0	0	1	0	1			
Remainder of A		0	0	0	0	0	0	0	1	1	1	1
Subtraction 6							0	1	0	1		
Result of subtraction 6						1	1	1	1	0		
Intermediate quotient					0	0	1	0	1	0		
Remainder of A		0	0	0	0	0	0	0	1	1	1	1
Subtraction 7								0	1	0	1	
Result of subtraction 7							0	0	0	1	0	
Intermediate quotient					0	0	1	0	1	0	1	
Remainder of A		0	0	0	0	0	0	0	0	1	0	1
Subtraction 8									0	1	0	1
Result of subtraction 8								0	0	0	0	0

Quotient Q					0	0	1	0	1	0	1	1
Final remainder R									0	0	0	0

The quotient of this division is $Q = (00101011)_2 = (43)_{10}$ with a remainder $R = (0000)_2 = (0)_{10}$. We can thus write $A = Q \times B + R = (00101011)_2 \times (0101)_2 + (0000)_2 = (43)_{10} \times (5)_{10} + (0)_{10}$.

• Two's complement representation of signed binary numbers

It is often necessary to perform arithmetic operations on signed numbers, i.e. numbers that can be positive or negative. In this case, the most significant bit (MSB) is always the sign bit: if $MSB = 0$, the number is positive, whereas, if $MSB = 1$, the number is negative.

The most common way to represent a signed decimal number X using an n -bit number is the two's complement representation.

The two's complement representation is different from the natural binary representation previously considered for unsigned numbers. It works as follows:

- If a decimal X number is positive or equal to zero ($X \geq 0$), its two's complement binary representation is identical to its natural binary representation.

The strict requirement that $MSB = 0$ implies that the amplitude, i.e. the absolute value $|X|$, of X is represented by the $(n - 1)$ rightmost bits. In other words, the positive decimal values of X can only range from 0 to $2^{n-1} - 1$. For instance, for $n = 3$, the positive decimal values of X can only be 0, +1, +2, or +3.

- To obtain the two's complement binary representation of a negative decimal number X ($X < 0$), we need to compute the two's complement of the natural representation of its absolute value $|X|$. When doing so, we must end up with a MSB equal to one, which will indicate that the number is negative. Once again, this implies that the amplitude (absolute value) of X is only represented by the $(n - 1)$ rightmost bits. The negative decimal values

of X can thus range from -1 to -2^{n-1} . For instance, for $n = 3$, the negative decimal values of X can only be -1 , -2 , -3 , or -4 .

As an illustration, by applying this procedure for $n = 3$, we obtain the table shown below.

Decimal number X	Absolute value $ X $ of X	Natural binary representation of $ X $	Two's complement representation of X
+3	3	011	011
+2	2	010	010
+1	1	001	001
0	0	000	000
-1	1	001	111
-2	2	010	110
-3	3	011	101
-4	4	100	100

There is an alternative way to explain how to represent a decimal number X in two's complement:

- If $X \geq 0$, we use the natural binary representation of the decimal number X , exactly as we do with unsigned numbers. For n -bit numbers, the leftmost bit must be equal to zero, thus leaving the $(n - 1)$ rightmost bits to represent the absolute value. When dealing with integer numbers, those numbers can thus range in decimal from 0 to $2^{n-1} - 1$.

- If $X < 0$, the two's complement representation of X is identical to the natural binary representation of the decimal number $2^n + X = 2^n - |X|$. For n -bit numbers, the leftmost bit must be equal to one, thus leaving the $(n - 1)$ rightmost numbers to represent the absolute value $|X|$. Since the latter can vary from 1 to 2^{n-1} in decimal, we can write

$$1 \leq |X| \leq 2^{n-1},$$

which is equivalent to

$$-2^{n-1} \leq -|X| \leq -1.$$

This inequality can be written as

$$2^n - 2^{n-1} \leq 2^n - |X| \leq 2^n - 1,$$

thus yielding

$$2^{n-1} \leq 2^n - |X| \leq 2^n - 1.$$

This inequality means that all natural binary representations of $2^n - |X|$, which represent all two's complement representations of the negative number X , have MSBs equal to one. For instance, for $n = 4$, it has just been shown that we can encode in two's complement all negative values ranging from $-2^{n-1} = -8$ to -1 . This requires the use of all natural binary representations of the decimal numbers ranging from $2^{n-1} = 8$ to $2^n - 1 = 15$. It does work perfectly since the natural representation of these numbers has an MSB equal to one.

The table below summarises the important point to remember. It is in fact pretty simple.

Decimal number X	2s complement of X = natural representation of...
$X \geq 0$	X
$X < 0$	$2^n - X $ or $2^n + X$

As an illustration of this alternative method, we obtain the table shown below in the case where $n = 3$.

Decimal number X	Natural binary representation of X	$2^n + X$ $= 8 + X$	Natural binary representation of $8 + X$	Signed binary number using two's complement representation
+3	011	-	-	011
+2	010	-	-	010
+1	001	-	-	001
0	000	-	-	000
-1	-	7	111	111
-2	-	6	110	110
-3	-	5	101	101
-4	-	4	100	100

As another illustration, we have shown, in the table below, the 16 signed values that can be generated by using four bits with the two's complement representation. For each binary number, we have also shown its equivalent in decimal.

4-bit signed number	Decimal number
0111	+7
0110	+6
0101	+5
0100	+4
0011	+3
0010	+2
0001	+1
0000	0
1111	-1
1110	-2
1101	-3
1100	-4
1011	-5
1010	-6
1001	-7
1000	-8

In order to be complete, one should mention that there exists a mathematical definition of two's complement. This expression can be used to determine the decimal number associated with any given two's complement representation. In two's complement, a binary number $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ does represent the decimal number

$$A = -a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0 = -a_{n-1}2^{n-1} + \sum_{i=0}^{n-2} a_i2^i.$$

This expression looks remarkably similar to that defining the natural binary representation:

$$A = \sum_{i=0}^{n-1} a_i2^i = a_{n-1}2^{n-1} + \sum_{i=0}^{n-2} a_i2^i.$$

The only difference between both expressions is that, in two's complement, the weight of the MSB (sign bit) is actually negative instead of being positive.

The similarity between both expressions explains why arithmetic operations in two's complement and natural can be performed by using the same basic computation algorithms, as will be seen later on.

In these expressions, one can notice that, when the MSB a_{n-1} is zero, the equations become strictly identical, which is why two's complement and natural representations are identical for positive numbers.

However, if the MSB a_{n-1} is one, then the difference in decimal value between both representations is equal to $2^{n-1} + 2^{n-1} = 2^n$. For instance, the decimal equivalent of $(1001)_2$ is $(9)_{10}$ in natural and $(-7)_{10}$ in two's complement, which corresponds to a difference equal to $2^4 = 16$. This confirms the fact that the two's complement of a negative decimal number A is the natural representation of $2^n + A$.

• Addition and subtraction on signed binary numbers

With signed arithmetic, there is no longer any real need to consider subtractions as subtractions turn out to be additions; a subtraction $(A - B)$ being indeed identical to the addition $A + (-B)$.

Let us now consider several examples of additions on 4-bit signed numbers to check how it all works and make sure we understand how to proceed for each possible configuration.

- $A = (+2)_{10} = (0010)_2$ and $B = (+3)_{10} = (0011)_2$.

Carry			0	1	0	0
A			0	0	1	0
B			0	0	1	1
A + B		0	0	1	0	1
Weight		2^n	2^{n-1}			

We obtain $A + B = (0010)_2 + (0011)_2 = (00101)_2$, which is equivalent in decimal to $(+2)_{10} + (+3)_{10} = (+5)_{10}$. This result is clearly correct.

Before starting, let us all agree on something fundamental: the sign bit is the bit of weight 2^{n-1} . Mathematics dictate that it should be the bit of weight 2^n . Unfortunately, this is a luxury one cannot afford in practical digital systems because all input and output numbers

must follow the same format for consistency reasons. It is crucial to stick to that rule religiously if we want our system to function properly and avoid complications later on. For instance, in a system operating on n -bit numbers, there is generally no room to store an $(n + 1)$ -th bit.

A good side of this rule is that the bit of weight 2^n can be discarded as it then no longer carries any relevant information.

We thus have $A + B = (0010)_2 + (0011)_2 = (0101)_2$. The result is correct and the sign bit (weight 2^{n-1}) is equal to zero, which is consistent with the fact that the sum of two positive numbers must also be positive. The bit of weight 2^n is ignored.

- $A = (+6)_{10} = (0110)_2$ and $B = (+3)_{10} = (0011)_2$.

Carry			1	1	0	0
A			0	1	1	0
B			0	0	1	1
A + B		0	1	0	0	1
Weight		2^n	2^{n-1}			

We obtain $A + B = (0110)_2 + (0011)_2 = (1001)_2$, which is equivalent in decimal to $(+6)_{10} + (+3)_{10} = (+9)_{10}$. This result is wrong.

In two's complement arithmetic on n -bit numbers, the bit of weight 2^{n-1} is, by convention, the sign bit for all numbers in order to stay consistent among input and output numbers. This comes in contradiction with mathematics which state that the MSB (here, the bit of weight 2^n) is the sign bit in the sum of two n -bit numbers.

Therefore, the result here is wrong as the value of the bit of weight 2^{n-1} falsely indicates that the sum of two positive numbers is negative. The issue here is that we have exceeded the maximum capacity ($= +7$) of our 4-bit representation, on the positive side, since $A + B = +9$, which is greater than $+7$.

If we allowed the bit of weight 2^n to be the sign bit, as nature intended, the result would be interpreted as $(01001)_2$, which is equivalent in decimal to +9. It would work perfectly fine in this case. So, we understand that the issue that we have encountered does not originate from some inherent limitations of two's complement arithmetic. Instead, it is entirely due to our arbitrary, but justified, decision to designate the sign bit as the bit of weight 2^{n-1} instead of the bit of weight 2^n .

Conclusion: When adding two (positive) n -bit numbers with bits of weight 2^{n-1} equal to zero, the bit of weight 2^{n-1} in the sum must also be equal to zero. If so, the result is correct. If this is not the case, the result is NOT correct because the maximal capacity of the representation has been exceeded.

Note that the bit of weight 2^n in the sum is always equal to zero, no matter what, unlike what happens with unsigned numbers. It does make sense as this is, as a matter of fact, the true sign bit.

- $A = (-2)_{10} = (1110)_2$ and $B = (-3)_{10} = (1101)_2$.

Carry			1	0	0	0
A			1	1	1	0
B			1	1	0	1
A + B		1	1	0	1	1
Weight		2^n	2^{n-1}			

Since A and B are both negative, their two's complement binary representations have been obtained by using the natural representations of the decimal numbers $2^n + A$ and $2^n + B$. In other words, the binary sum that has been computed here is the natural representation of the decimal number $2^n + 2^n + A + B$.

When 2^n is subtracted from this result (meaning that the bit of weight 2^n , which is equal to one, is set to zero or, equivalently, discarded), the result that is left corresponds to the natural representation of the number $2^n + A + B$. This is precisely what we want because

this is, by definition, the two's complement binary representation of the negative decimal number $A + B$.

We obtain $A + B = (1110)_2 + (1101)_2 = (11011)_2$. After discarding the leftmost bit, the result of the sum is $(1011)_2$. So, in decimal, the sum is $(-2)_{10} + (-3)_{10} = (-5)_{10}$. This result is clearly correct.

- $A = (-7)_{10} = (1001)_2$ and $B = (-3)_{10} = (1101)_2$.

Carry			0	0	1	0
-7			1	0	0	1
-3			1	1	0	1
Sum		1	0	1	1	0
Weight		2^n	2^{n-1}			

After discarding the leftmost bit, the result must be the natural binary representation of the decimal number $2^n + A + B$, that is the sum $A + B$ in two's complement. Here, it is observed that the sign bit, i.e. the bit of weight 2^{n-1} , equals zero. The result cannot be correct because this sign bit should be equal to one as the sum of two negative numbers must also be negative. The reason behind this error is simply that the maximum capacity (-8) of the representation has been exceeded on the negative side since $A + B = -10 < -8$.

That is what is observed here as $(1001)_2 + (1101)_2 = (10110)_2$. After discarding the leftmost bit, we obtain the result $(0110)_2$. In decimal, this is equivalent to having $(-7)_{10} + (-3)_{10} = (+6)_{10}$, which is obviously a wrong result.

Once again, let us remember that, if the sign bit had been chosen to be the bit of weight 2^n , the result would have then been interpreted as $(10110)_2$, which is equivalent in decimal to -10 . It would have been the correct result. Hence, let us understand once for all that the issue only arises from our arbitrary, but necessary, decision to choose the bit of weight 2^{n-1} instead of the bit 2^n as the sign bit. It comes by no means from inherent limitations of two's complement arithmetic. This is a good example of a conflict between mathematical theory and practical implementation.

Conclusion: When adding two negative n -bit numbers, with bits of weight 2^{n-1} obviously equal to one, the bit of weight 2^{n-1} of the sum must also be equal to one. If so, the result is correct. If this is not the case, the result is NOT correct because this means that the maximal capacity of the representation has been exceeded.

Note that the bit of weight 2^n in the sum always equals one. It does make sense as this is in fact the true sign bit in mathematical terms. With the convention adopted earlier, this bit must be discarded as, in two's complement, any bit left of the sign bit is irrelevant.

- $A = (-2)_{10} = (1110)_2$ and $B = (+7)_{10} = (0111)_2$.

Carry			1	1	0	0
A			1	1	1	0
B			0	1	1	1
A + B		1	0	1	0	1
Weight		2^n	2^{n-1}			

This time, we have $A < 0$ and $B > 0$ with $|A| < |B|$, meaning that $A + B$ is a positive number. The issue of going out of range, on either the positive or negative side, does no longer exist in such configuration. Since $A < 0$ and $B \geq 0$, we have in fact computed the natural binary representation of the decimal number $2^n + A + B$.

Once the bit of weight 2^n is discarded, the result that is left corresponds to the natural representation of the positive decimal number $A + B$. The result is thus correct.

It is observed here that $(1110)_2 + (0111)_2 = (10101)_2$. After discarding the leftmost bit, we obtain the result $(0101)_2$. In decimal, this is equivalent to having $(-2)_{10} + (+7)_{10} = (+5)_{10}$, which is indeed the correct result.

For those who like to dig a bit deeper, would the result still be correct if we kept the bit of weight 2^n as the sign bit and thus did not discard it?

We have seen that the addition generated the natural representation of the decimal number $2^n + A + B$. If 2^n is subtracted from this result, which is equivalent to inverting the leftmost bit of weight 2^n from one to zero, the result that is left corresponds to the natural representation of the positive decimal number $A + B$. This is the desired result. We conclude that the simple inversion of the sign bit would have provided a final, and correct, result equal to $(00101)_2$.

- $A = (+2)_{10} = (0010)_2$ and $B = (-7)_{10} = (1001)_2$.

Carry			0	0	0	0
A			0	0	1	0
B			1	0	0	1
A + B		0	1	0	1	1
Weight		2^n	2^{n-1}			

Once again, A and B do not have the same sign, but this time we have $A \geq 0$ and $B < 0$ with $|A| < |B|$, meaning that $A + B$ is a negative number. Since $A \geq 0$ and $B < 0$, we have in fact computed here the natural binary representation of the decimal number $2^n + A + B$.

As such, the result is thus the two's complement representation of the negative decimal number $A + B$, which is precisely what is desired. The result is thus correct as it stands.

In this example, the sum $(0010)_2 + (1001)_2 = (01011)_2$. Ignoring the leftmost bit as usual, we obtain the result $(1011)_2$. The bit of weight 2^{n-1} is equal to one, thus indicating that it is a negative number. In decimal, this is equivalent to having $(+2)_{10} + (-7)_{10} = (-5)_{10}$, which is indeed the correct result.

For our intellectual satisfaction, how to proceed to obtain the correct result if the bit of weight 2^n was adopted as the sign bit?

The raw result is $(01011)_2$ and that is clearly not the negative number that one expects. We have earlier shown that the result is the natural representation of the decimal number $2^n + A + B$. This can also be written as $2^{n+1} - 2^n + A + B$. By adding 2^n to that number, i.e.

inverting the leftmost bit of weight 2^n from zero to one, we are left with $2^{n+1} + A + B$. That is the natural representation of the negative number $A + B$ using $(n + 1)$ bits. We conclude that the simple inversion of the sign bit would have provided a final, and correct, result equal to $(11011)_2$.

Conclusion: When adding two n -bit numbers with opposite signs, the result is always correct. Note that the maximal capacity of the representation, on either the positive or negative side, cannot be exceeded under this scenario. In other words, nothing can go wrong when summing two numbers with opposite signs.

We have summarised in the table below the various conclusions reached in this section regarding the sum of two signed numbers A and B using the two's complement representation. Note that this table is only valid under the following convention: the bit of weight 2^{n-1} is adopted as the sign bit in the sum result. Under this convention, the bit of weight 2^n is always discarded.

Bit of weight 2^{n-1} in A (sign bit)	Bit of weight 2^{n-1} in B (sign bit)	Other condition	Bit of weight 2^{n-1} in $S = A + B$ (sign bit)	Bit of weight 2^n in $S = A + B$ (sign bit)	Correct result if...
0	0	-	0 or 1	0	Bit of weight $2^{n-1} = 0$ (else result out-of-range)
1	1	-	0 or 1	1	Bit of weight $2^{n-1} = 1$ (else result out-of-range)
0	1	$A + B \geq 0$	0	1	-
0	1	$A + B < 0$	1	0	-

In order to be thorough, we have also summarised in the table below the conclusions reached when the bit of weight 2^n is allowed to be the sign bit, as mathematics intended. Those results are probably of little interest in most practical digital systems.

Bit of weight 2^{n-1} in A (sign bit)	Bit of weight 2^{n-1} in B (sign bit)	Other condition	Bit of weight 2^n in $S = A + B$ (sign bit)	Correct result if...
0	0	-	0	-
1	1	-	1	-
0	1	$A + B \geq 0$	1	Bit of weight 2^n inverted ($1 \rightarrow 0$)
0	1	$A + B < 0$	0	Bit of weight 2^n inverted ($0 \rightarrow 1$)

At this stage, we can ask ourselves another question: could we reach the same conclusions by only relying on the generic “mathematical” definition of two’s complement?

Recall that a given n -bit number $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ does represent in two’s complement the decimal number

$$A_{2s} = -a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0 = -a_{n-1}2^{n-1} + \sum_{i=0}^{n-2} a_i2^i,$$

whereas it corresponds in natural representation to another decimal number

$$A_{nat} = a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0 = a_{n-1}2^{n-1} + \sum_{i=0}^{n-2} a_i2^i.$$

We can easily show that $A_{nat} = a_{n-1}2^{n-1} + A_{2s} + a_{n-1}2^{n-1} = a_{n-1}2^n + A_{2s}$.

For instance, assuming $n = 4$, we have $A_{nat} = 16 \cdot a_{n-1} + A_{2s}$. Hence, if $A_{2s} = -7$, then we can write $A_{nat} = 9$. This simply means that the two’s complement representation of the decimal number -7 is identical to the natural representation of the decimal number 9.

Now, let us consider the addition of two n -bit numbers $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ and $(b_{n-1}b_{n-2} \dots b_1b_0)_2$. The result of this addition is an $(n + 1)$ -bit number $(s_ns_{n-1}s_{n-2} \dots s_1s_0)_2$.

It is worth mentioning that, in a digital system operating on n -bit numbers, there is generally no room to accommodate the $(n + 1)$ -th bit s_n . In the case of n -bit signed numbers, the sign bit is the bit s_{n-1} and the bit s_n can actually be discarded. If, for any reason, it is not possible to safely discard s_n , then a warning flag must be activated to inform the system about this. This is often done by setting an overflow bit to one.

If the result $(s_n s_{n-1} s_{n-2} \dots s_1 s_0)_2$ of the addition is seen as a natural binary representation, it then corresponds to a decimal number S_{nat} given by $S_{nat} = A_{nat} + B_{nat}$.

If this number is seen as a two's complement representation, it then corresponds to a decimal number S_{2s} given by

$$S_{2s} = S_{nat} - s_n 2^{n+1} = A_{nat} + B_{nat} - s_n 2^{n+1},$$

which is equivalent to

$$S_{2s} = a_{n-1} 2^n + A_{2s} + b_{n-1} 2^n + B_{2s} - s_n 2^{n+1}.$$

We finally obtain $S_{2s} = A_{2s} + B_{2s} + a_{n-1} 2^n + b_{n-1} 2^n - s_n 2^{n+1}$.

This shows that, by summing in the traditional way two n -bit numbers represented in two's complement, the result of this addition does not directly yield the desired result, $S_{2s} = A_{2s} + B_{2s}$. Instead, it generates the result $S_{2s} = A_{2s} + B_{2s} + (a_{n-1} + b_{n-1} - 2s_n) 2^n$.

To obtain the sum $S_{2s} = A_{2s} + B_{2s}$ of two integers represented in two's complement, one must simply subtract the term $(a_{n-1} + b_{n-1} - 2s_n) 2^n$ from the raw result produced by this addition.

Assume first that bit s_n is the sign bit. There are four possibilities described in the table below.

a_{n-1}	b_{n-1}	Other condition	s_n	Value of $(a_{n-1} + b_{n-1} - 2s_n) 2^n$	How to correct the raw result to obtain $S_{2s} = A_{2s} + B_{2s}$?
0	0	-	0	0	-
1	1	-	1	0	-
$a_{n-1} \neq b_{n-1}$		$S_{2s} \geq 0$	0	2^n	Subtract $2^n \rightarrow$ Invert s_n (1 $\rightarrow$ 0)
		$S_{2s} < 0$	1	-2^n	Add $2^n \rightarrow$ Invert s_n (0 $\rightarrow$ 1)

One can notice that the results are in complete agreement with those found in the previous analysis. It has, in fact, been very straightforward to obtain them using the equation $S_{2s} = A_{2s} + B_{2s} + (a_{n-1} + b_{n-1} - 2s_n)2^n$.

Assume now that bit s_{n-1} is the sign bit. This is the most important scenario as it corresponds to the convention adopted in most digital designs. In two's complement, any bit beyond the sign bit is irrelevant and must thus be discarded. The additional term, $(a_{n-1} + b_{n-1} - 2s_n)2^n$, is associated with the weight 2^n and therefore irrelevant in an n -bit number in two's complement. It can de facto be discarded, and we are simply left with the expression $S_{2s} = A_{2s} + B_{2s}$.

The only thing to do consists of making sure that the result makes sense, which is not always the case.

Scenario (a): Assume $A_{2s} \geq 0$ and $B_{2s} \geq 0$ ($a_{n-1} = b_{n-1} = 0$). Since the sum S_{2s} must be positive, the result can only be valid if $s_{n-1} = 0$. As $a_{n-1} = b_{n-1} = 0$, this happens when the carry generated by the weight- 2^{n-2} stage in the addition is also zero. In other words, the positive sum S_{2s} must not exceed the value $2^{n-1} - 1$ for the result to be valid. This is consistent with the fact that the maximum decimal value that can be generated with an n -bit number in two's complement is $2^{n-1} - 1$.

a_{n-1}	b_{n-1}	s_{n-1}	Comments
0	0	0 or 1	$s_{n-1} = 0 \rightarrow$ Correct result, no modification needed $s_{n-1} = 1 \rightarrow$ Wrong result (out of range)

Scenario (b): $A_{2s} < 0$ and $B_{2s} < 0$ ($a_{n-1} = b_{n-1} = 1$). Since S_{2s} must be negative, the result can only make sense if $s_{n-1} = 1$. Given the fact that $a_{n-1} = b_{n-1} = 1$, this occurs only when the carry generated by the weight- 2^{n-2} stage in the addition equals one. This is equivalent to stating that the negative sum S_{2s} must be greater than or equal to -2^{n-1} . Note that this result was demonstrated earlier in the introductory chapter on two's complement.

a_{n-1}	b_{n-1}	s_{n-1}	Comments
0	0	0 or 1	$s_{n-1} = 0 \rightarrow$ Correct result, no modification needed $s_{n-1} = 1 \rightarrow$ Wrong result (out of range)
1	1	0 or 1	$s_{n-1} = 1 \rightarrow$ Correct result, no modification needed $s_{n-1} = 0 \rightarrow$ Wrong result (out of range)

Scenario (c): $A_{2s} \geq 0$ and $B_{2s} < 0$ ($a_{n-1} = 0$ and $b_{n-1} = 1$). As a positive number is added to a negative number under this scenario, it is easy to see that there cannot be any out-of-range issue, unlike in scenarios (a) and (b). Indeed, as we have $0 \leq A_{2s} \leq 2^{n-1} - 1$ and $-2^{n-1} \leq B_{2s} < 0$, we can write

$$-2^{n-1} \leq S_{2s} \leq 2^{n-1} - 1.$$

Since S_{2s} can be positive or negative, the sign bit s_{n-1} can take both values zero and one.

a_{n-1}	b_{n-1}	s_{n-1}	Comments
0	0	0 or 1	$s_{n-1} = 0 \rightarrow$ Correct result, no modification needed $s_{n-1} = 1 \rightarrow$ Wrong result (out of range)
1	1	0 or 1	$s_{n-1} = 1 \rightarrow$ Correct result, no modification needed $s_{n-1} = 0 \rightarrow$ Wrong result (out of range)
$a_{n-1} \neq b_{n-1}$		0 or 1	Correct result, no modification needed

We have performed a thorough and mathematically rigorous investigation of the addition operation in the context of two's complement. Our reasoning has relied upon the mathematical definition of this binary representation. All conclusions previously reached have been confirmed.

• Sign Extension of Signed Quantities

Consider an n -bit vector $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ representing a quantity in two's complement. It is often required to extend this number to m bits, with $m > n$. This typically happens when one needs to perform an arithmetic operation using an n -bit number A and an m -bit quantity B.

If $A \geq 0$, the rather obvious thing to do consists of appending $(m - n)$ zeros to the n -bit number. For instance, if $A = (0101)_2$ and $B = (11010011)_2$, we must use the number $A = (00000101)_2$ for any arithmetic operation with B . It makes perfect sense as $(0101)_2$ and $(00000101)_2$ do clearly represent the same quantity.

If $A < 0$, one must instead append $(m - n)$ ones to the n -bit number. For instance, if $A = (1101)_2$ and $B = (10111011)_2$, we must use the number $A = (11111101)_2$ for any arithmetic operation with B .

In other words, the sign bit must simply be extended whether A is positive or negative.

It is worth demonstrating this result for the case $A < 0$.

When $A < 0$, the two's complement encoding of A on n bits is a binary number corresponding to the natural representation of the decimal quantity $2^n + A$.

By appending $(m - n)$ ones to this n -bit number, we now obtain the natural representation of the quantity

$$2^{m-1} + 2^{m-2} + \dots + 2^{n+1} + 2^n + 2^n + A = \sum_{i=n}^{m-1} 2^i + 2^n + A = 2^m + A,$$

which is still the two's complement encoding of the same number A , but using m bits rather than n bits.

To understand that demonstration, one must remember that, for any integers m and n , with $n < m$, we have

$$2^m - 1 = \sum_{i=0}^{m-1} 2^i,$$

which can also be written as

$$2^m - 1 = \sum_{i=0}^{n-1} 2^i + \sum_{i=n}^{m-1} 2^i = 2^n - 1 + \sum_{i=n}^{m-1} 2^i.$$

Hence, we finally have $2^m = 2^n + \sum_{i=n}^{m-1} 2^i$.

• Multiplication on signed binary numbers

We have extensively studied the issue of summing two's complement binary numbers. Our conclusions have been that the sum of two such numbers can be calculated by using the standard addition procedure originally devised for natural binary representation.

What about the multiplication of two's complement numbers? Can one employ the standard multiplication procedure to multiply such numbers?

Let us consider the multiplication of two n -bit numbers $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ and $(b_{n-1}b_{n-2} \dots b_1b_0)_2$.

The result of this multiplication is a $2n$ -bit number $(p_{2n-1}p_{2n-2}p_{2n-3} \dots p_1p_0)_2$.

Assume first that these numbers are represented in natural binary and thus correspond to the decimal numbers

$$A_{nat} = a_{n-1}2^{n-1} + a_{n-2}2^{n-2} + \dots + a_12^1 + a_02^0$$

and

$$B_{nat} = b_{n-1}2^{n-1} + b_{n-2}2^{n-2} + \dots + b_12^1 + b_02^0.$$

From now on, we are going to consider the particular case $n = 4$ as an illustration. Hence, we can write $A_{nat} = a_32^3 + a_22^2 + a_12^1 + a_02^0$ and $B_{nat} = b_32^3 + b_22^2 + b_12^1 + b_02^0$.

The standard multiplication procedure generates the decimal product

$$\begin{aligned} P_{nat} &= A_{nat} \times B_{nat} = (a_32^3 + a_22^2 + a_12^1 + a_02^0)(b_32^3 + b_22^2 + b_12^1 + b_02^0) \\ \rightarrow P_{nat} &= a_3b_32^6 + (a_3b_2 + a_2b_3)2^5 + (a_3b_1 + a_2b_2 + a_1b_3)2^4 + (a_3b_0 + a_2b_1 + a_1b_2 + \\ &\quad a_0b_3)2^3 + (a_2b_0 + a_1b_1 + a_0b_2)2^2 + (a_1b_0 + a_0b_1)2^1 + a_0b_02^0. \end{aligned}$$

The table below illustrates how the multiplication is performed on natural binary numbers. First, the $4 \times 4 = 16$ partial products $p_{i,j} = a_ib_j, i \in \{0, 1, 2, 3\}$, are computed. Then, these partial products have to be summed to generate the bits $p_i, i \in \{0, 1, \dots, 7\}$.

							a_3	a_2	a_1	a_0
$\times$							b_3	b_2	b_1	b_0
							a_3b_0	a_2b_0	a_1b_0	a_0b_0
$+$							a_3b_1	a_2b_1	a_1b_1	a_0b_1
$+$						a_3b_2	a_2b_2	a_1b_2	a_0b_2	.
$+$					a_3b_3	a_2b_3	a_1b_3	a_0b_3	.	.
Product			p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0
Weight			2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

Assume now that these binary numbers are represented in two's complement rather than in natural binary. In this case, $(a_{n-1}a_{n-2} \dots a_1a_0)_2$ and $(b_{n-1}b_{n-2} \dots b_1b_0)_2$ correspond to the decimal numbers A_{2s} and B_{2s} , respectively, with $A_{2s} = A_{nat} - a_{n-1}2^n$ and $B_{2s} = B_{nat} - b_{n-1}2^n$.

The question is as follows: can we compute the product $P_{2s} = A_{2s} \times B_{2s}$ by using the result produced by the multiplication process described above?

We have

$$P_{2s} = P_{nat} - p_{2n-1}2^{2n} = A_{nat}B_{nat} - p_{2n-1}2^{2n},$$

which leads to

$$P_{2s} = (A_{2s} + a_{n-1}2^n)(B_{2s} + b_{n-1}2^n) - p_{2n-1}2^{2n}.$$

which is equivalent to

$$P_{2s} = A_{2s}B_{2s} + b_{n-1}A_{nat}2^n + a_{n-1}B_{nat}2^n + (a_{n-1}b_{n-1} - p_{2n-1})2^{2n}.$$

In these expressions, the bits a_{n-1} , b_{n-1} , and p_{2n-1} denote the sign bits of A_{2s} , B_{2s} , and P_{2s} , respectively.

According to this expression, the desired product, $P_{2s} = A_{2s}B_{2s}$, is not available at the end of the traditional multiplication process. However, it is possible to compute it by subtracting three terms from the result produced by this process, namely $b_{n-1}A_{nat}2^n$, $a_{n-1}B_{nat}2^n$, and $(a_{n-1}b_{n-1} - p_{2n-1})2^{2n}$.

Remarkably, the third term, $(a_{n-1}b_{n-1} - p_{2n-1})2^{2n}$, can be ignored because its weight is 2^{2n} , which is beyond that of the sign bit. Remember that, in two's complement, any bit left of the sign bit can be discarded.

For our example with $n = 4$, the desired product, $P_{2s} = A_{2s}B_{2s}$, is obtained by subtracting, from the raw result generated by the multiplication process,

1. the 4-bit number $(a_3b_3 \ a_2b_3 \ a_1b_3 \ a_0b_3)_2$ shifted by four positions;
2. the 4-bit number $(a_3b_3 \ a_3b_2 \ a_3b_1 \ a_3b_0)_2$ shifted by four positions.

Note that there cannot be any out-of-range issue with a multiplication in two's complement. In fact, we know that both operands are such that

$$-2^{n-1} \leq A_{2s} \leq 2^{n-1} - 1 \text{ and } -2^{n-1} \leq B_{2s} \leq 2^{n-1} - 1.$$

Therefore, the final product $P_{2s} = A_{2s}B_{2s}$ is inherently bounded as

$$-2^{2n-2} + 2^{n-1} \leq P_{2s} \leq 2^{2n-2},$$

which easily fits within the $[-2^{2n-1}, 2^{2n-1} - 1]$ range achievable with a $2n$ -bit number in two's complement.

The table below summarizes the various steps needed to multiply two 4-bit numbers in two's complement. The whole process seems more complicated than that used for numbers in natural binary.

							a_3	a_2	a_1	a_0
$\times$							b_3	b_2	b_1	b_0
							a_3b_0	a_2b_0	a_1b_0	a_0b_0
+						a_3b_1	a_2b_1	a_1b_1	a_0b_1	.
+					a_3b_2	a_2b_2	a_1b_2	a_0b_2	.	.
+				a_3b_3	a_2b_3	a_1b_3	a_0b_3	.	.	.
−			a_3b_3	a_2b_3	a_1b_3	a_0b_3	.	.	.	.
−			a_3b_3	a_3b_2	a_3b_1	a_3b_0	.	.	.	.
Product			p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0
Weight		2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

It is, however, possible to simplify it by replacing both subtractions by additions, and then moving bits and logic ones around in a not-so-clever way, as shown below.

$$P_{2s} = (\overline{a_3b_3} + \overline{a_3b_3})2^7 + (a_3b_3 + \overline{a_2b_3} + \overline{a_3b_2})2^6 + (a_3b_2 + a_2b_3 + \overline{a_1b_3} + \overline{a_3b_1})2^5 + (a_3b_1 + a_2b_2 + a_1b_3 + \overline{a_0b_3} + \overline{a_3b_0} + 1 + 1)2^4 + (a_3b_0 + a_2b_1 + a_1b_2 + a_0b_3)2^3 + p_22^2 + p_12^1 + p_02^0$$

							a_3	a_2	a_1	a_0
$\times$							b_3	b_2	b_1	b_0
							a_3b_0	a_2b_0	a_1b_0	a_0b_0
+						a_3b_1	a_2b_1	a_1b_1	a_0b_1	.
+						a_3b_2	a_2b_2	a_1b_2	a_0b_2	.
+				a_3b_3	a_2b_3	a_1b_3	a_0b_3	.	.	.
+			$\overline{a_3b_3}$	$\overline{a_2b_3}$	$\overline{a_1b_3}$	$\overline{a_0b_3}$	.	.	.	.
+			$\overline{a_3b_3}$	$\overline{a_3b_2}$	$\overline{a_3b_1}$	$\overline{a_3b_0}$	.	.	.	.
						1				
						1				
Product			p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0
Weight		2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$$\rightarrow P_{2s} = (0)2^7 + (a_3b_3)2^6 + (a_3b_2 + a_2b_3 + 2\overline{a_2b_3} + 2\overline{a_3b_2})2^5 + (a_3b_1 + a_2b_2 + a_1b_3 + 2\overline{a_1b_3} + 2\overline{a_3b_1} + 2)2^4 + (a_3b_0 + a_2b_1 + a_1b_2 + a_0b_3 + 2\overline{a_0b_3} + 2\overline{a_3b_0})2^3 + p_22^2 + p_12^1 + p_02^0$$

							a_3	a_2	a_1	a_0
$\times$							b_3	b_2	b_1	b_0
							a_3b_0	a_2b_0	a_1b_0	a_0b_0
+						a_3b_1	a_2b_1	a_1b_1	a_0b_1	.
+					a_3b_2	a_2b_2	a_1b_2	a_0b_2	.	.
+					a_2b_3	a_1b_3	a_0b_3	.	.	.
+					$2\overline{a_2b_3}$	$2\overline{a_1b_3}$	$2\overline{a_0b_3}$	.	.	.

+	$\overline{a_3b_3}$		a_3b_3	$2\overline{a_3b_2}$	$2\overline{a_3b_1}$	$2\overline{a_3b_0}$	.	.	.	
					2					
Product		p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0	
Weight		2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$$\rightarrow P_{2s} = (0)2^7 + (a_3b_3)2^6 + (\overline{a_2b_3} + 1 + \overline{a_3b_2} + 1)2^5 + (a_2b_2 + \overline{a_1b_3} + 1 + \overline{a_3b_1} + 1 + 2)2^4 + (a_2b_1 + a_1b_2 + \overline{a_0b_3} + 1 + \overline{a_3b_0} + 1)2^3 + p_22^2 + p_12^1 + p_02^0$$

							a_3	a_2	a_1	a_0
×							b_3	b_2	b_1	b_0
							$\overline{a_3b_0}$	a_2b_0	a_1b_0	a_0b_0
+						$\overline{a_3b_1}$	a_2b_1	a_1b_1	a_0b_1	.
+					$\overline{a_2b_3}$	a_2b_2	a_1b_2	a_0b_2	.	.
+					$\overline{a_3b_2}$	$\overline{a_1b_3}$	$\overline{a_0b_3}$	.	.	.
+					1	1	1	.	.	.
+				a_3b_3	1	1	1	.	.	.
						2				
Product			p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0
Weight		2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

$$\rightarrow P_{2s} = (1)2^7 + (a_3b_3)2^6 + (\overline{a_2b_3} + \overline{a_3b_2})2^5 + (a_2b_2 + \overline{a_1b_3} + \overline{a_3b_1} + 1)2^4 + (a_2b_1 + a_1b_2 + \overline{a_0b_3} + \overline{a_3b_0})2^3 + p_22^2 + p_12^1 + p_02^0$$

							a_3	a_2	a_1	a_0
×							b_3	b_2	b_1	b_0
						1	$\overline{a_3b_0}$	a_2b_0	a_1b_0	a_0b_0
+						$\overline{a_3b_1}$	a_2b_1	a_1b_1	a_0b_1	.
+					$\overline{a_2b_3}$	a_2b_2	a_1b_2	a_0b_2	.	.
+			1	a_3b_3	$\overline{a_3b_2}$	$\overline{a_1b_3}$	$\overline{a_0b_3}$	.	.	.
Product			p_7	p_6	p_5	p_4	p_3	p_2	p_1	p_0

Weight		2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
--------	--	-------	-------	-------	-------	-------	-------	-------	-------	-------

We have been able to simplify the multiplication process to a point where it can be considered as being of similar complexity as that for natural numbers.

Consider a few examples.

- $A = (+6)_{10} = (0110)_2$ and $B = (+3)_{10} = (0011)_2$.

						0	1	1	0
×						0	0	1	1
Carries		1	1,1	1,1	1,1	1			
					1	$\bar{0} = 1$	1	1	0
+					$\bar{0} = 1$	1	1	0	.
+				$\bar{0} = 1$	0	0	0	.	.
+		1	0	$\bar{0} = 1$	$\bar{0} = 1$	$\bar{0} = 1$	.	.	.
Product		0	0	0	1	0	0	1	0
Weight		2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

We find $P = (00010010)_2 = (+18)_{10}$. Correct!

- $A = (-7)_{10} = (1001)_2$ and $B = (-3)_{10} = (1101)_2$.

						1	0	0	1
×						1	1	0	1
Carries		1	1	1					
					1	$\bar{1} = 0$	0	0	1
+					$\bar{0} = 1$	0	0	0	.
+				$\bar{1} = 0$	0	0	1	.	.
+		1	1	$\bar{0} = 1$	$\bar{0} = 1$	$\bar{1} = 0$	.	.	.
Product		0	0	0	1	0	1	0	1
Weight		2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

We find $P = (00010101)_2 = (+21)_{10}$. Correct!

- $A = (-7)_{10} = (1001)_2$ and $B = (+7)_{10} = (0111)_2$.

						1	0	0	1
×						0	1	1	1
Carries			1	1					
					1	$\bar{1} = 0$	0	0	1
+					$\bar{1} = 0$	0	0	1	.
+				$\bar{1} = 0$	0	0	1	.	.
+		1	0	$\bar{0} = 1$	$\bar{0} = 1$	$\bar{0} = 1$	.	.	.
Product		1	1	0	0	1	1	1	1
Weight		2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

We find $P = (11001111)_2 = (-49)_{10}$. Correct!

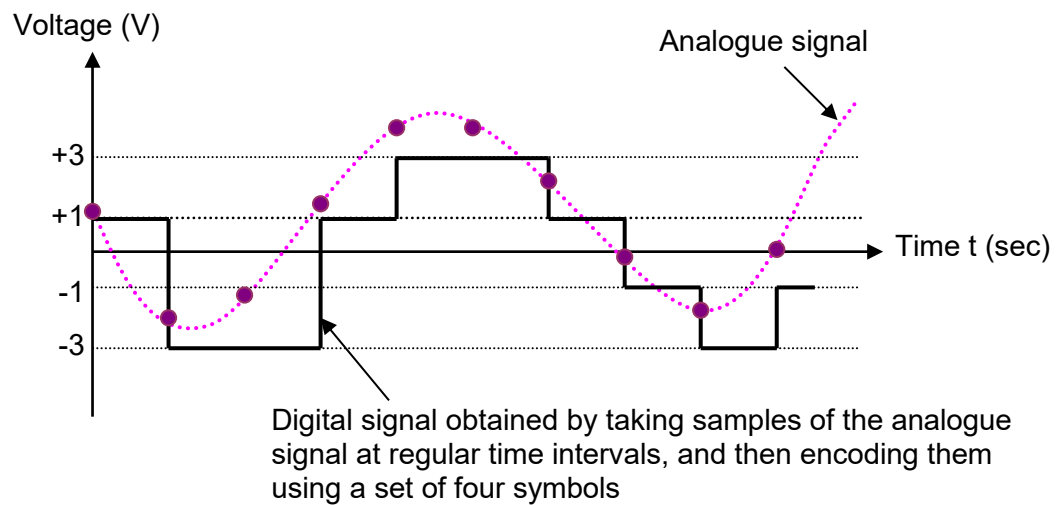
• Representing Binary Quantities

In digital systems, any type of information, beyond numbers, is presented in binary form. Binary quantities can be represented by any device that has only two operating states or possible conditions, for instance by a switch (either open or closed).

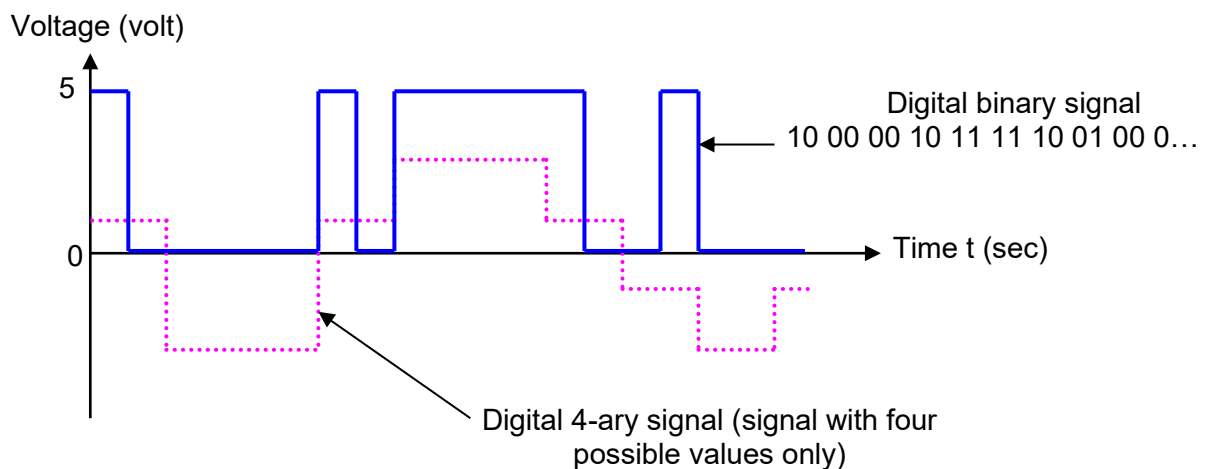
A typical voltage assignment in an electronic circuit is as follows:

- A logic 1 is represented by any voltage value equal or close to the supply voltage of the circuit;
- A logic 0 is represented by any voltage value equal or close to zero volt (ground).

As an illustration, an analogue signal can be converted into a digital signal with an arbitrary number of levels:



Generally, the number of levels is a power of two so that the digital signal can be further converted into a binary digital signal, i.e. a string of bits.



• Decimal-to-Binary Conversion

As we have already seen, the binary system is employed in all digital systems. Thus, decimal values must be converted to binary values before they are entered into a digital system.

How to convert a decimal integer number into binary, based on the binary natural representation?

We can think of breaking the decimal number into a sum of powers of two, as shown in this example:

$$\begin{aligned}(45)_{10} &= 32 + 0 + 8 + 4 + 0 + 1 \\ &= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= (101101)_2\end{aligned}$$

Alternatively, we can divide the decimal number by two and record the remainder. Then, we subtract this remainder from the original number, divide by two the number thus obtained, and repeat the procedure until we reach zero. An example for the decimal number $(45)_{10}$ is as follows:

- $(45)_{10}/2 = 2 \times 22 + 1 \rightarrow \text{Remainder} = 1 \rightarrow \text{New decimal number} = (45)_{10} - 1 = (44)_{10}$
- Divide $(44)_{10}$ by 2 $\rightarrow (22)_{10} \rightarrow (22)_{10}/2 = 2 \times 11 + 0 \rightarrow \text{Remainder} = 0 \rightarrow \text{New decimal number} = (22)_{10} - 0 = (22)_{10}$
- Divide $(22)_{10}$ by 2 $\rightarrow (11)_{10} \rightarrow (11)_{10}/2 = 2 \times 5 + 1 \rightarrow \text{Remainder} = 1 \rightarrow \text{New decimal number} = (11)_{10} - 1 = (10)_{10}$
- Divide $(10)_{10}$ by 2 $\rightarrow (5)_{10} \rightarrow (5)_{10}/2 = 2 \times 2 + 1 \rightarrow \text{Remainder} = 1 \rightarrow \text{New decimal number} = (5)_{10} - 1 = (4)_{10}$
- Divide $(4)_{10}$ by 2 $\rightarrow (2)_{10} \rightarrow (2)_{10}/2 = 2 \times 1 + 0 \rightarrow \text{Remainder} = 0 \rightarrow \text{New decimal number} = (2)_{10} - 0 = (2)_{10}$
- Divide $(2)_{10}$ by 2 $\rightarrow (1)_{10} \rightarrow (1)_{10}/2 = 2 \times 0 + 1 \rightarrow \text{Remainder} = 1 \rightarrow \text{New decimal number} = (1)_{10} - 1 = (0)_{10} \rightarrow \text{End of the algorithm.}$

So, by tracing back the successive remainders, we conclude that $(45)_{10} = (101101)_2$.

• Binary-to-Decimal Conversion

This type of conversion is straightforward. An example is as follows:

$$\begin{aligned}(10110101)_2 &= 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ &= 128 + 0 + 32 + 16 + 0 + 4 + 0 + 1 \\ &= (181)_{10}\end{aligned}$$